



Gymnázium J. K. Tyla

Úvod do agentického AI a harness pro automatizovaný softwarový vývoj

ODBORNÁ PRÁCE

Autor práce: Patrik Marius, 4.D
Vedoucí práce: Michal Dočekal

Rozsah práce: 777 slov / 5340 znaků

Prohlášení

Prohlašuji, že jsem tuto studentskou odbornou práci vypracoval/a samostatně pod dohledem vedoucího uvedeného na první straně. Všechny použité zdroje jsou uvedeny v seznamu zdrojů a informace z nich získané jsou v textu řádně označeny odkazem na zdroj. Souhlasím s tím, aby tištěná forma práce byla uchována na Gymnázium J. K. Tyla a tam používána jako tištěný zdroj např. pro další studentské práce či pro prezentaci vzdělávání na GJKT.

V Hradci Králové dne

Podpis autora práce:

.....

Anotace

Tato odborná práce se zabývá principy agentního inženýrství (*agentic engineering*) a architekturou řídicích harnessů pro automatizovaný vývoj softwaru. Praktickým přínosem práce je návrh a implementace systému DarkFactory — agentního harnessu instalovatelného jako aplikace pro platformu GitHub (GitHub App). Systém usiluje o maximální možnou míru automatizace vývojového cyklu od sémantické analýzy požadavků v GitHub Issues, přes technické plánování, až po generování kódu a vystavení pull requestu. Práce reflektuje, že současné agentní systémy nelze vnímat jako plně autonomní: jazykové modely vyžadují deterministické mantinely proti uvíznutí v nekonečných cyklech, správu kontextu bez sémantického posunu a především kontinuální zapojení člověka formou schvalovacích bran (*Human-in-the-loop*).

Annotation

This thesis explores the foundational principles of agentic engineering and harness architecture for automated software development. The practical contribution of the work is the design and implementation of DarkFactory — an agentic harness installable as a GitHub App. The system aims to maximize the automation of the software engineering lifecycle, ranging from issue interpretation in GitHub Issues and technical planning to code generation and pull request delivery. The thesis emphasizes that modern agentic systems cannot be considered fully autonomous: large language models require deterministic guardrails against infinite loops, rigorous context management without semantic drift, and indispensable human-in-the-loop governance via approval gates.

Klíčová slova | Keywords

Agent (Agent), Agent Loop (Agentní smyčka), Chatbot (Chatbot), Context Rot (Degradace kontextu), Embedding (Vektorová reprezentace), Git (Git), GitHub (GitHub), Harness (Řídicí harness), Human-in-the-loop (Zapojení člověka do smyčky), MCP (MCP), Plugins (Zásuvné moduly), Prompt Engineering (Promptové inženýrství), Pull Request (Pull Request), Skills (Dovednosti)

Agent (Agent)

[EN] A software system driven by a language model and equipped with tools, capable of independently planning, observing the state of its environment, and carrying out multi-step actions toward a specified goal.

[CZ] Softwarový systém řízený jazykovým modelem a vybavený nástroji, který samostatně plánuje, vnímá stav prostředí a provádí vícezkrokové akce směřující k dosažení zadaného cíle.

Agent Loop (Agentní smyčka)

[EN] An iterative execution cycle of an autonomous agent, based on the ReAct pattern (Reasoning + Acting), in which the model alternates between reasoning, tool calls, and evaluation of observations from the runtime environment.

[CZ] Iterativní prováděcí cyklus autonomního agenta (založený na vzoru ReAct: Reasoning + Acting), v němž model střídavě uvažuje, volá nástroje a vyhodnocuje pozorování z běhového prostředí.

Chatbot (Chatbot)

[EN] A language-model-based system designed primarily for text interaction with a user; it responds to conversational requests but does not by itself provide an autonomous execution loop for modifying the surrounding environment.

[CZ] Systém založený na jazykovém modelu určený především k textové interakci s uživatelem; odpovídá na požadavky v konverzaci, ale sám o sobě neposkytuje autonomní prováděcí smyčku pro změny okolního prostředí.

Context Rot (Degradace kontextu)

[EN] A practical term for degradation in a model's ability to reliably use relevant information as the context becomes long, noisy, or internally competing, which can lead to missed instructions and lost relationships between facts.

[CZ] Praktické označení pro zhoršování schopnosti modelu spolehlivě využívat relevantní informace při dlouhém, zašuměném nebo vzájemně si konkurujícím kontextu, což může vést k přehlížení instrukcí a ztrátě souvislostí.

Embedding (Vektorová reprezentace)

[EN] A multidimensional vector representation used to encode tokens or other data so that numerical operations can capture useful relationships between representations.

[CZ] Vícerozměrná vektorová reprezentace tokenů nebo jiných dat, která umožňuje numerickými operacemi zachycovat užitečné vztahy mezi reprezentacemi.

Git (Git)

[EN] A distributed version-control system for recording source-history changes, branching, merging, and returning to earlier repository states.

[CZ] Distribuovaný systém správy verzí umožňující sledování historie změn kódu, větvení, slučování změn a návrat k předchozím stavům repozitáře.

GitHub (GitHub)

[EN] A platform for hosting Git repositories and coordinating software-development workflows such as Issues, Pull Requests, and CI/CD automation.

[CZ] Platforma pro hosting gitových repozitářů a koordinaci vývojového cyklu pomocí funkcí jako Issues, Pull Requests a automatizace CI/CD.

Harness (Řídicí harness)

[EN] The application and orchestration layer surrounding a model's inference core; it provides tools, context management, guardrails, state handling, and control over the execution lifecycle.

[CZ] Aplikační a orchestrační vrstva obklopující inferenční jádro modelu; zajišťuje nástroje, správu kontextu, bezpečnostní mantinely, práci se stavem a řízení prováděcího životního cyklu.

Human-in-the-loop (Zapojení člověka do smyčky)

[EN] A design pattern in which a human operator remains part of the system's decision process, for example through approval gates before selected consequential actions.

[CZ] Návrhový vzor, v němž lidský operátor zůstává součástí rozhodovacího procesu systému, například prostřednictvím schvalovacích bran před vybranými významnými akcemi.

MCP (MCP)

[EN] Model Context Protocol, an open protocol for connecting AI applications to external tools, resources, and data sources through standardized interfaces.

[CZ] Model Context Protocol, otevřený protokol pro připojování AI aplikací k externím nástrojům, zdrojům a datům prostřednictvím standardizovaných rozhraní.

Plugins (Zásuvné moduly)

[EN] Programmatic extension modules that add specialized adapters, tools, or deterministic execution behavior to the harness runtime.

[CZ] Programové rozšiřující moduly, které do běhového prostředí harnessu přidávají specializované adaptéry, nástroje nebo deterministické prováděcí chování.

Prompt Engineering (Promptové inženýrství)

[EN] The systematic design and structuring of instructions and prompts used to shape and constrain the behavior of a language model.

[CZ] Systematický návrh a strukturování instrukcí a promptů používaných k usměrnění a omezení chování jazykového modelu.

Pull Request (Pull Request)

[EN] A formal proposal to integrate changes from one repository branch into another, providing a place for automated checks, human review, and discussion.

[CZ] Formální návrh na začlenění změn z jedné větve repozitáře do druhé, který poskytuje prostor pro automatizované kontroly, lidskou revizi a diskusi.

Skills (Dovednosti)

[EN] Reusable packages of instructions, procedural rules, and optional helper resources that a harness can load into an agent's context for a particular class of task.

[CZ] Znovupoužitelné balíčky instrukcí, procedurálních pravidel a případných pomocných zdrojů, které může harness načíst do kontextu agenta pro určitý typ úlohy.

Obsah

Prohlášení	2
Anotace	3
Annotation	4
Klíčová slova Keywords	5
1 Úvod	10
1.1 Motivace a vymezení problému	10
1.2 Cíl práce a výzkumné otázky	10
1.3 Metodika práce	10
2 Teoretická část – Analýza konceptu	11
2.1 Git a GitHub (Správa verzí)	11
2.1.1 Úvod	11
2.2 LLM, chatboti a agenti	11
2.2.1 Úvod	11
2.2.2 Tokeny, tokenizace a embedding	12
2.3 Harness a agentní inženýrství (prompt, kontext, smyčka, graf...)	12
2.3.1 Úvod	12
2.3.2 Agent vs. Chatbot	12
3 Praktická část – Návrh architektury	14
4 Výsledky a diskuse	15
5 Závěr	16
Seznam příloh	17
Seznam zdrojů	18

Seznam obrázků a tabulek

1 Úvod

1.1 Motivace a vymezení problému

1.2 Cíl práce a výzkumné otázky

Hlavní cíl: Vymezit teoretické principy agentního inženýrství (*agentic engineering*) a navrhnout modulární architekturu řídicího harnessu pro automatizovaný vývoj softwaru se zachováním lidského dohledu v klíčových rozhodovacích bodech.

Dílčí cíle:

- Vymezit infrastrukturu pro správu verzí (Git, GitHub a kontinuální integraci).
- Analyzovat limity velkých jazykových modelů (dynamiku kontextového okna, jev Context Rot, ztrátovou kompresi a sémantický posun).
- Navrhnout architekturu řídicího harnessu zahrnující nástrojové smyčky (ReAct), bezpečnostní pískoviště a hierarchickou orchestraci subagentů.
- Formalizovat mechanismy zapojení člověka do smyčky (*Human-in-the-loop*), schvalovací brány a protokol revizních značek pro dohled nad textovými výstupy.

1.3 Metodika práce

Práce má teoreticko-architektonický a inženýrský charakter. Vzhledem k dynamickému vývoji v oblasti autonomního softwarového vývoje práce důsledně zachovává a integruje zavedené anglické odborné názvy (např. *harness*, *pull request*, *agent loop*, *prompt engineering*, *skills* či *context rot*). Použití této terminologie je integrální součástí práce, neboť tyto anglické pojmy představují de facto celosvětové průmyslové standardy (*industry standards*), jejichž doslovný český překlad by byl nejednoznačný, zavádějící či v rozporu s běžnou inženýrskou praxí.

Postup práce sleduje strukturu inženýrského cyklu:

- **1. Analýza konceptu:** Systematické zmapování limitů autoregresivních modelů, dynamiky kontextového okna, jevu Context Rot a rozhraní nástrojů.
- **2. Návrh architektury:** Formulace modulárního modelu řídicího harnessu, správy stavu, exekučního pískoviště, bezpečnostních pojistek a orchestrace subagentů.
- **3. Kritické zhodnocení:** Porovnání navržených principů s volnými agentními smyčkami a vymezení provozních limitů autonomního inženýrství.

2 Teoretická část – Analýza konceptu

2.1 Git a GitHub (Správa verzí)

2.1.1 Úvod

Pro autonomní vývoj softwaru je spolehlivá správa verzí naprosto nezbytným základem. Jazykové modely generují kód na základě statistické pravděpodobnosti, a proto se nevyhnutelně dopouštějí chyb, logických přehmatů či regresí. Verzovací systém vytváří bezpečné a deterministické prostředí, v němž lze každou úpravu zaznamenat, otestovat a v případě selhání kdykoliv vrátit zpět k funkčnímu stavu. Namísto teoretických abstrakcí práce přímo využívá distribuovaný systém **Git** v kombinaci s platformou **GitHub**.

Klíčové komponenty infrastruktury zahrnují:

- **Distribuovaný systém Git** (1): Ukládá kompletní historii projektu v podobě jednotlivých revizí (*commitů*). Vývojář i agent pracují s plnou lokální kopií repozitáře, což umožňuje provádět změny, přepínat větve a spouštět lokální testy zcela nezávisle na síťovém připojení.
- **Platforma GitHub**: Slouží jako centrální bod pro sdílení kódu, týmovou koordinaci a automatizaci:
 - **Zadávání a sledování úkolů (Issues)**: Strukturovaná textová zadání požadavků a hlášení chyb, která agentovi slouží jako výchozí specifikace úlohy.
 - **Revize změn (Pull Requests)**: Uživatelské rozhraní pro přehledné zobrazení diffu, diskusi nad kódem a formální schválení člověkem.
 - **Automatizace (GitHub Actions)**: Běhové prostředí pro automatické spouštění testů, linterů a překladů při každé události v repozitáři.

Agent v tomto pojetí nevystupuje jako černá skříňka s proprietárním protokolem, nýbrž jako standardní přispěvatel, který plně respektuje běžné vývojářské zvyklosti a nástroje.

2.2 LLM, chatboti a agenti

2.2.1 Úvod

V agentním softwarovém inženýrství vystupuje velký jazykový model (LLM) jako stochastické kognitivní jádro celého systému. Z hlediska vnitřní architektury se jedná o dekodérový transformer (*Decoder-only*), jehož typickými představiteli jsou moderní modely řad Claude, GPT či DeepSeek (2). Role modelu nespočívá ve vystupování jako vševědoucí orákulum se spolehlivou znalostí okolního světa, nýbrž jako pokročilý generátor hypotéz, kódu a strukturovaných volání nástrojů řízený obdrženým kontextem.

Základní principy fungování modelu zahrnují:

- **Autoregresivní predikce**: Model zpracovává zadanou sekvenci textu a na jejím základě iterativně předpovídá nejpravděpodobnější následující symboly (*tokens*).
- **Stochastická povaha**: Vzhledem k pravděpodobnostnímu vzorkování může model na totožný vstup reagovat mírně odlišně, což vyžaduje deterministické mantinely v nadřazeném řídicím harnessu.

Pro efektivní nasazení modelu do vývojového cyklu je nezbytné porozumět způsobu, jakým reprezentuje informace a jaké fyzické limity vymezují jeho operační paměť.

2.2.2 Tokeny, tokenizace a embedding

Jazykový model nepracuje přímo se znaky ani slovy v lidském slova smyslu. Vstupní text je nejprve deterministickým algoritmem převeden na číselné reprezentace, se kterými následně počítají maticové vrstvy neuronové sítě.

Tento proces zahrnuje následující pojmy:

- **Tokeny a tokenizér:** Token představuje základní diskrétní jednotku (celé slovo, slabiku či fragment znaků). Převod mezi textem a posloupností číselných tokenů zajišťuje tokenizér (nejčastěji na bázi algoritmu Byte-Pair Encoding, BPE).
- **Embedding:** Každý token je promítnut do vícerozměrného vektorového prostoru, kde geometrická vzdálenost a úhel vektorů vyjadřují sémantickou příbuznost pojmů (např. vektorová analogie král – muž + žena $\approx$ královna).
- **Jazyková asymetrie tokenizace:** Vzhledem k trénovacím datům optimalizovaným primárně pro angličtinu spotřebovávají flektivní jazyky s bohatou diakritikou (včetně češtiny) 2× až 3× více tokenů pro vyjádření téhož významu.

Z inženýrského hlediska je proto žádoucí vést systémové prompty, technické plány i komunikaci mezi nástroji v angličtině, aby se šetřila kapacita kontextu a snížila latence inference.

2.3 Harness a agentní inženýrství (prompt, kontext, smyčka, graf...)

2.3.1 Úvod

V terminologii agentního inženýrství označuje pojem **Harness** aplikační vrstvu, která obklopuje samotné inferenční jádro jazykového modelu. Samotné inferenční jádro provádí výhradně matematické maticové operace nad zadanými váhami a vektory tokenů; veškerou orchestraci, práci se soubory a řízení bezpečnosti zajišťuje harness.

Ústřední komponentou a hlavní prováděcí funkcí, která v architektuře harnessu řídí samotný běh a iterativní koordinaci agenta v reálném vývojovém prostředí, je takzvaná **Agent Loop** (agentní smyčka).

2.3.2 Agent vs. Chatbot

Rozdíl mezi konverzačním **Chatbot**em a autonomním **Agent**em nespočívá v odlišném jazykovém modelu, ale v architektuře jeho zapojení do pracovního prostředí. Zatímco chatbot funguje pasivně jako textový rádce, agent vystupuje jako aktivní vykonavatel úkolů.

Srovnání obou přístupů:

- **Konverzační chatbot:**
 - Reaguje pouze na přímé textové výzvy v uzavřeném okně chatu.
 - Nemá přímý přístup k souborovému systému ani k nástrojům operačního systému.
 - Uživatel musí navržený kód ručně zkopírovat, vložit do projektu a otestovat.
- **Autonomní agent:**
 - Je vybaven sadou výkonných nástrojů (*tools*) pro práci s repozitářem.
 - Aktivně prozkoumává soubory, modifikuje zdrojový kód, spouští testy a interpretuje jejich návratové kódy.

- Funguje v autonomní prováděcí smyčce, v níž iterativně reaguje na reálnou odezvu vývojového prostředí.

3 Praktická část – Návrh architektury

4 Výsledky a diskuse

5 Závěr

Seznam příloh

Seznam zdrojů

1. CHACON, Scott and STRAUB, Ben. *Pro Git*. 2. New York : Apress, 2014. ISBN 978-1-4842-0076-6.
2. VASWANI, Ashish, SHAZEER, Noam, PARMAR, Niki, USZKOREIT, Jakob, JONES, Llion, GOMEZ, Aidan N., KAISER, Łukasz and POLOSUKHIN, Illia. Attention Is All You Need. *arXiv preprint arXiv:1706.03762*. Online. 2017. Available from: <https://arxiv.org/abs/1706.03762>
3. MARIUS, Patrik. DarkFactory: autonomous, governed software engineering pipelines. Online. 2026. [Accessed 8 September 2026]. Available from: <https://github.com/marius-patrik/DarkFactory>
4. HUMBLE, Jez and FARLEY, David. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Boston : Addison-Wesley, 2010. ISBN 978-0-321-60191-9.
5. Deepseek Harness. *arXiv preprint arXiv:2608.25512*. Online. 2026. Available from: <https://arxiv.org/abs/2608.25512>
6. ANTHROPIC. Model Context Protocol documentation. Online. 2026. Available from: <https://modelcontextprotocol.io/docs/2026-07-28/getting-started/intro#explore-mcp>
7. ANTHROPIC. Prompt Engineering overview. Online. 2026. Available from: <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview>
8. YAO, Shunyu, ZHAO, Jeffrey, YU, Dian, DU, Nan, SHAFRAN, Izhak, NARASIMHAN, Karthik and CAO, Yuan. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629*. Online. 2022. Available from: <https://arxiv.org/abs/2210.03629>
9. LIU, Nelson F., LIN, Kevin, HEWITT, John, PARANJAPE, Ashwin, BEVILACQUA, Michele, PETRONI, Fabio and LIANG, Percy. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*. Online. 2024. Vol. 12, p. 157–173. Available from: <https://arxiv.org/abs/2307.03172>
10. DAO, Tri, FU, Daniel Y., ERMON, Stefano, RUDRA, Atri and RÉ, Christopher. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *Advances in Neural Information Processing Systems*. Online. 2022. Vol. 35, p. 16344–16359. Available from: <https://arxiv.org/abs/2205.14135>
11. AINSLIE, Joshua, LEE-THORP, James, JONG, Michiel de, ZEMLYANSKIY, Yury, LEBRÓN, Federico and SANGHAI, Sumit. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. *arXiv preprint arXiv:2305.13245*. Online. 2023. Available from: <https://arxiv.org/abs/2305.13245>